Demo: Integrating Pydantic for Data Validation in FastAPI

1. Overview

In this demo, learners will enhance an existing **ETL API pipeline** by adding **data validation using Pydantic**. Validation ensures that incoming data follows predefined rules before it is processed by the system.

Using **FastAPI and Pydantic**, invalid data automatically triggers a **422 validation response**, clearly indicating which fields failed validation and why. This makes APIs safer, more reliable, and self-documented.

2. Learning Objectives

By the end of this demo, learners will be able to:

- Implement **data validation using Pydantic models**
- Validate uploaded data before processing it
- Return **structured validation errors**
- Prevent corrupted data from entering the ETL pipeline
- Build **self-documented APIs using schemas**

3. Project Setup

The project contains the following files:

```
project/
│
├── main.py
```

```
├── requirements.txt
├── clean_data.csv
└── corrupted_data.csv
```

File Description

main.py

Contains the FastAPI application and validation logic.

requirements.txt

Contains all project dependencies.

clean_data.csv

Dataset containing valid records.

corrupted_data.csv

Dataset containing invalid records used to demonstrate validation errors.

4. Dependencies

Install the required libraries.

```
fastapi
uvicorn
pandas
python-multipart
pydantic
```

Install them using:

```
pip install -r requirements.txt
```

5. Running the Application

Start the FastAPI server using **Uvicorn**.

```
uvicorn main:app --reload
```

Open the API documentation:

http://127.0.0.1:8000/docs

Swagger UI will show all available API endpoints.

6. Pydantic Schema for Data Validation

We define a **Pydantic model** to validate each record in the uploaded dataset.

```
from pydantic import BaseModel, Field

class PersonRecord(BaseModel):

    name: str = Field(..., min_length=1)

    age: int = Field(..., ge=18, le=100)

    city: str = Field(..., min_length=1)

    salary: float = Field(..., ge=0)
```

Validation Rules

Field	Rule

name	Cannot be empty
age	Must be between 18 and 100
city	Cannot be empty
salary	Must be greater than or equal to 0

These rules ensure that the ETL pipeline processes **consistent and clean data**.

7. Validator Helper Function

A helper function validates each row of the dataset against the schema.

```
def validate_records(df):

    valid_rows = []
    errors = []

    for index, row in df.iterrows():
        try:
            record = PersonRecord(**row.to_dict())
            valid_rows.append(record.dict())
        except Exception as e:
            errors.append({"row": index, "error": str(e)})

    return valid_rows, errors
```

Function Behavior

- Converts each row into a **Pydantic object**
- Valid rows are added to the dataset
- Invalid rows return **detailed error messages**

8. Extract CSV with Validation

The API endpoint uploads a CSV file and validates each record.

```
from fastapi import FastAPI, UploadFile, File
import pandas as pd
import uuid

app = FastAPI()

data_store = {}


@app.post("/extract-csv-validated")
async def extract_csv_validated(file: UploadFile = File(...)):

    contents = await file.read()

    df = pd.read_csv(pd.io.common.BytesIO(contents))

    valid_rows, errors = validate_records(df)

    if len(valid_rows) == 0:
        return {"message": "No valid rows after validation", "errors":
errors}

    clean_df = pd.DataFrame(valid_rows)

    token = str(uuid.uuid4())

    data_store[token] = clean_df

    return {
        "token": token,
        "preview": clean_df.head().to_dict(orient="records"),
        "errors": errors
    }
```

What this API does

1. Uploads a CSV file
2. Converts the file into a **Pandas DataFrame**
3. Validates each row using **Pydantic**
4. Stores only **valid rows**
5. Returns a **token for further processing**

9. Testing Validation with Invalid Data

Upload the file:

`corrupted_data.csv`

Example issues in the dataset:

- Missing values
- Negative salary
- Empty name
- Incorrect age

Response example:

```
{
 "message": "No valid rows after validation"
}
```

This demonstrates how validation **prevents invalid data from entering the ETL pipeline**.

10. Uploading Valid Data

Next, upload the **clean dataset**.

```
clean_data.csv
```

Response example:

```
{
 "token": "abc123",
 "preview": [
   {"name":"Vivek","age":35,"city":"Bangalore","salary":60000}
 ]
}
```

The returned **token** will be used for transformation and loading steps.

11. Applying Transformation

The **transform API** works the same as in the previous ETL demo.

Example request:

```
{
 "token":"abc123",
 "select":["name","age","city"],
 "rename":{"age":"years"}
}
```

If incorrect filter syntax is used, FastAPI returns a **clear error message** explaining the issue.

12. Loading Data to SQLite

Finally, the cleaned and transformed data can be stored in the database.

```
@app.post("/load")
def load_data(token: str, table_name: str):

    df = data_store[token]

    conn = sqlite3.connect("etl.db")

    df.to_sql(table_name, conn, if_exists="replace", index=False)

    conn.close()

    return {"status": "Data loaded successfully"}
```

The data is now stored in the **SQLite database**.

13. Key Takeaways

- **Pydantic validation** ensures only valid data enters the system
- FastAPI automatically generates **clear validation error responses**
- Validation improves **data quality and pipeline reliability**
- The ETL pipeline becomes **predictable and easier to debug**
- APIs remain **self-documented through schema definitions**